

MATLAB C-V Measurement Guideline

Zurich Instruments MFIA

Author: R. Aliasgari Renani

Date: 13 May 2026

Basic Use and Code Behavior

1.1 Purpose

This document is a short lab reference for the MATLAB App Designer app `CV_Zurich_3.mlapp`. The app connects to a Zurich Instruments MFIA through LabOne, configures the MFIA impedance analyzer, sweeps the DC bias voltage, and saves measured voltage and capacitance data to an Excel file.

The C-V measurement is done by applying a small AC impedance excitation at a fixed frequency while sweeping a DC voltage across the device under test, for example a diode or MOS capacitor. In the current code, the swept voltage is the Signal Output offset and the saved capacitance is taken from `/imps/<imps_channel>/sample` as `sample.param1`.

1.2 Basic Use

- Start LabOne and make sure the MFIA is visible to the LabOne backend.
- The current app is hardcoded for device ID `dev5478`, API level `6`, `127.0.0.1:8006`, and PCIe connection.
- Enter sample information, comments, save folder, impedance frequency, excitation amplitude, impedance model, voltage sweep limits, point count, settling, averaging, and bandwidth settings.
- Press **Start Experiment**. The app disables existing settings, configures the MFIA, runs the Sweeper, updates the plots, and then asks where to save the data.
- The Excel output has a Settings sheet and a Measurements sheet. The measurement columns are voltage and capacitance.

1.3 Important Code Behavior

- The swept node is `sigouts/<out_c>/offset`. Therefore `start_voltage` and `stop_voltage` are DC output-offset values.
- The app reads the actual output channel from `/imps/<imps_channel>/output/select`. The MFIA node documentation lists this node as read-only and gives Signal Output 1 as the available output.
- The app hardcodes the generic signal output range and the impedance output range to 10 V.
- The app forces impedance output on at the end of configuration with `/imps/<imps_channel>/output/on=1`.
- The app forces impedance input autorange on. The forced node/value is `/imps/<imps_channel>/auto/inputrange = 1`.
- The app enables internal calibration and disables user compensation.
- The app sets impedance current input and voltage input to input 0.
- The app saves `sample.param1` as capacitance. This is only correct when the selected impedance model has capacitance as representation parameter 2, such as an R-C representation.

Parameter Reference

2.1 Signal Input Parameters

- **in_c**: Signal Input channel index used for `/sigins/<in_c>` and generic demodulator input selection. Default in the UI is 0. The exact available channel list should be read from the MFIA node tree. Source: app code and MFIA node docs.
- **input_imp50**: Sets `/sigins/<in_c>/imp50`. Allowed values are 0 for 10 MOhm and 1 for 50 Ohm. UI default is 1. For high-impedance C-V samples, check whether 50 Ohm loading is really intended. Source: MFIA node docs.
- **input_ac**: Sets both `/sigins/<in_c>/ac` and `/imps/<imps_channel>/ac`. Allowed values are 0 for DC coupling and 1 for AC coupling. UI default is 1. AC coupling inserts a high-pass filter. Source: MFIA node docs.
- **input_range**: Sets `/sigins/<in_c>/range` in volts. UI default is 1 V. The manual says the range should exceed the incoming signal by roughly a factor of two and that the instrument selects the next higher available range. Exact available range values were not clearly stated in the checked sources.

2.2 Signal Output Parameters

- **out_c**: Intended output channel. In the current code this is not really user-configurable. The app overwrites it with the read-only value from `/imps/<imps_channel>/output/select`. The checked MFIA node docs identify this as Signal Output 1.
- **sigout_on**: Sets `/sigouts/<out_c>/on` after setup. UI default is 1. Important: the impedance output is also forced on by the code, so this field does not fully control whether the MFIA drives

the DUT.

- **sigout_range**: Misleading in the current app. The UI default is 10 V and the value is saved, but the code ignores the entered value and hardcodes `/sigouts/<out_c>/range=10` and `/imps/<imps_channel>/output/range=10`.
- **sigout_amplitude**: Sets the generic signal-output mixer amplitude at `/sigouts/<out_c>/amplitudes/<out_mixer_c>`. UI default is 0.05 V. For this C-V app, `imps_amplitude` is the clearer impedance excitation amplitude.
- **sigout_mixer_enable**: Enables the selected generic signal-output mixer amplitude through `/sigouts/<out_c>/enables/<out_mixer_c>`. UI default is 1. This looks like a legacy lock-in style control and should be reviewed.

2.3 Demodulator Parameters

- **demod_phaseshift**: Applies phase shift in degrees to `/demods/*/phaseshift`. UI default is 0. The checked sources did not clearly state a numeric range. Normally leave at 0 unless a calibrated phase correction is required.
- **demod_order**: Sets generic demodulator filter order and impedance demodulator order. Allowed values are 1 to 8, corresponding to 6 dB/oct to 48 dB/oct. UI default is 4. Higher order affects settling time.
- **demod_c**: Generic demodulator index used for rate, harmonic, and enable. UI default is 0. Available demodulators depend on the MFIA options. Several other demodulator settings are applied to all demodulators, so this field is only a partial channel selector.
- **demod_rate**: Sets generic and impedance demodulator data rate in samples per second. UI default is 13000. The checked sources did not clearly state the numeric range, and Zurich notes that entered values may be approximated to supported values.
- **demod_harmonic**: Sets the reference harmonic for the generic and impedance demodulator. UI default is 1. Use 1 for ordinary fundamental C-V measurements unless there is a specific reason to measure a harmonic.
- **demod_enable**: Enables the selected generic demodulator. Allowed values are 0 off and 1 on. UI default is 1.
- **demod_tc**: Sets the generic and impedance demodulator time constant in seconds. UI default is 0.007 s. The checked sources did not clearly state the numeric range; the device exposes min/max time-constant properties in its node tree.

2.4 Oscillator Parameters

- **osc_c**: Oscillator index used by the generic demodulators and the impedance demodulator. Documented oscillator selections are 0 to 3, depending on installed options. UI default is 0.
- **osc_freq**: Sets /oscs/<osc_c>/freq. UI default is 3 MHz. The exact frequency range depends on the MFIA version/options. In this app, **imps_freq** is the more relevant C-V excitation frequency control.

2.5 Impedance Analyzer Parameters

- **imps_channel**: Impedance analyzer unit index. UI default is 0. The manual describes one impedance analyzer unit normally, and two only with the MF-MD option.
- **imps_amplitude**: Sets /imps/<imps_channel>/output/amplitude. UI default is 0.01 V. This is the main AC test-signal amplitude for C-V. The checked node docs did not clearly state the exact numeric range.
- **imps_freq**: Sets /imps/<imps_channel>/freq. UI default is 100 kHz. It is the impedance excitation frequency for the sweep. The exact range depends on the MFIA model/options.
- **imps_mode**: Sets impedance measurement mode. Allowed values are 0 for 4-terminal and 1 for 2-terminal. UI default is 1. The manual notes that 2-terminal mode can be useful for very high impedance, while 4-terminal mode removes series effects when wiring supports it.
- **imps_model**: Selects the impedance representation model. Important values for C-V are 0 for R_p parallel C_p and 1 for $R_s + C_s$. The full documented list is 0 to 10. The app saves `sample.param1`, so use a model where parameter 2 is capacitance.

2.6 Sweep Parameters

- **start_voltage**: Sweeper start value for sigouts/<out_c>/offset. UI default is -0.5 V. This is the starting DC bias. The safe range depends on the MFIA output range, fixture, wiring, and DUT.
- **stop_voltage**: Sweeper stop value for sigouts/<out_c>/offset. UI default is 0.5 V. This is the final DC bias.
- **sweep_mode_scan**: Sweeper scan mode. Allowed values are 0 sequential, 1 binary, 2 bidirectional, and 3 reverse. UI default is 2. Bidirectional mode is useful for checking hysteresis.
- **sweep_samplecount**: Number of sweep points. UI default is 100. The checked Sweeper docs did not clearly state the maximum value.

- **sweep_inaccuracy**: Sweeper settling inaccuracy. Official Sweeper docs state the range as 1e-13 to 0.1. UI default is 0.001. Smaller values wait longer after each voltage step.
- **xmapping**: Sweep point spacing. Allowed values are 0 linear and 1 logarithmic. UI default is 0. Use linear for normal voltage sweeps, especially if the sweep crosses 0 V.
- **loopcount**: Number of sweeps to perform. UI default is 1. More than one loop can test repeatability, but the current saved data format is not ideal for multiple loops.
- **averaging_tc**: Minimum averaging time in demodulator time constants per sweep point. UI default is 50. Larger values reduce noise but increase measurement time.
- **averaging_sample**: Minimum number of samples per sweep point. UI default is 0. Zurich recommends enough samples for reliable statistics; leaving this at 0 makes averaging_tc dominate.
- **bandwidthcontrol**: Sweeper bandwidth mode. Allowed values are 0 manual, 1 fixed, and 2 automatic. UI default is 2. Automatic bandwidth can override manually chosen demodulator bandwidth settings.
- **bandwidthoverlap**: Allows bandwidth overlap between neighboring sweep points. UI default is 0. The exact numeric enumeration was not clearly stated in the checked Sweeper docs. This is more important for frequency sweeps than for a DC-bias C-V sweep.

2.7 Metadata and Saving

- **SampleInfo**: Free text saved in the settings sheet. Use it for sample ID, device type, contact, area, wafer, or location.
- **Comments**: Free text saved in the settings sheet. Use it for fixture, illumination, temperature, probe, and measurement notes.
- **DefaultFolder**: Save folder. It is restored from the MATLAB preference MyApp/DataFolder. If no preference exists, the app creates a MyAppData folder under the MATLAB user path.
- **additionalComments**: Optional extra comment added when using the **Save Data + Additional Comment** button after a measurement.
- **device_id**: Hardcoded private app property, currently dev5478. This is not user-facing and should probably become a device selection field later.

Chapter III

TODO

3.1 App Cleanup

- Disable or remove `out_c` as a user input because the code reads it from a read-only MFIA node.
- Disable or remove `sigout_range` because the code hardcodes 10 V.
- Clarify `sigout_on` because the impedance output is forced on.
- Decide whether `sigout_amplitude` and `sigout_mixer_enable` are still needed for this C-V workflow.
- Keep `imps_freq` as the main excitation-frequency control and avoid confusion with `osc_freq`.
- Add UI validation for voltage limits, positive point count, valid channel indices, valid model selection, and logarithmic sweep compatibility.
- Replace numeric 0/1 and enum fields with dropdowns or checkboxes.
- Save more raw MFIA data later, not only voltage and capacitance: impedance, model parameter 1, model parameter 2, frequency, drive amplitude, and confidence indicators if available.

3.2 DLTS Implementation Roadmap

For DLTS, the MFIA should be used as the capacitance-transient meter. The LakeShore controller handles temperature, and MATLAB should coordinate both instruments.

The extra instrument used in many older DLTS setups is usually a boxcar averager or DLTS correlator. It is not a second lock-in demodulator. Its job is to take the capacitance transient after a filling pulse and calculate a rate-window signal. We do not need that as separate hardware for the first version. We can record the full transient with the MFIA and calculate the DLTS signal later in MATLAB.

The MFIA already measures the small-signal capacitance. Zurich also shows DLTS examples where the MFIA records capacitance transients and can generate the bias pulse. An external pulse generator is only needed if the MFIA pulse is not good enough for our required voltage, timing, edge speed, or isolation.

Minimum setup:

- MFIA with LabOne API access from MATLAB.
- LakeShore temperature controller connected to MATLAB.
- Temperature stage or cryostat with stable wiring to the DUT.
- Low-parasitic fixture and compensation where needed.
- MATLAB code for transient averaging and rate-window calculation.

The current C-V app uses the LabOne Sweeper to step the DC output offset. DLTS should be a separate mode. It should not sweep voltage point by point. It should sit at one temperature, apply the same filling pulse many times, and record the capacitance recovery after each pulse.

Inputs needed for DLTS:

- reverse-bias voltage;
- filling-pulse voltage and pulse width;
- repetition period between pulses;
- transient length and DAQ sample rate;
- number of repeats to average;
- temperature start, stop, step, and stability tolerance;
- rate-window times for the DLTS signal.

One temperature point should run like this:

- Set the LakeShore temperature and wait until it is stable.
- Configure the MFIA capacitance measurement: frequency, AC amplitude, model, bandwidth, and data rate.
- Apply reverse bias and wait for the baseline capacitance to settle.
- Arm LabOne DAQ for the capacitance sample node.
- Apply the filling pulse and return to reverse bias.

- Record the capacitance transient after the pulse.
- Repeat and average the transient.
- Save the raw transients as well as the averaged transient.

Processing in MATLAB:

- Convert MFIA data to time, capacitance, temperature, and bias arrays.
- Reject bad traces: lost trigger, overload, clipping, contact jump, or unstable temperature.
- Average good traces at the same temperature.
- Calculate a simple DLTS signal first, for example $C(t_1) - C(t_2)$ or two averaged time windows.
- Plot DLTS signal versus temperature for each rate window.
- Use the peak positions later for emission-rate and Arrhenius analysis.
- Leave Laplace-DLTS for later. It needs cleaner data and more careful numerical work.

First tests before real samples:

- Check the pulse waveform before connecting a sensitive device.
- Test timing and triggering with a dummy capacitor or RC circuit.
- Repeat one temperature point many times and check drift/noise.
- Run a short temperature scan on a known diode or Schottky sample.
- Compare the capacitance level with ordinary C-V results.

For DLTS, saving only the final capacitance values is not enough. Each run should save raw transients, averaged transients, temperature history, reverse bias, pulse voltage, pulse width, repetition period, MFIA settings, DAQ settings, and selected rate windows.

Practical implementation order:

1. Add a separate `runDLTSExperiment` function.
2. Reuse the MFIA connection/setup code.
3. Replace the sweeper with triggered DAQ acquisition.
4. Add the LakeShore temperature loop.
5. Save raw transients first.
6. Add simple two-point DLTS processing.

7. Add plots for transient, averaged transient, and DLTS signal versus temperature.